IDOR: SANTA'S LITTLE IDOR

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	4. Common Web Vulnerabilities
Platform	TryHackMe
Room	IDOR - Santa's Little IDOR (https://tryhackme.com/room/idor-aoc2025-zl6MywQid9)
Focus	IDOR, authentication vs authorization, horizontal privilege escalation
Completion	100%
Date	October 2026

1. Objective

Understand IDOR (Insecure Direct Object Reference): how to spot it, how to exploit it to read other users' data, and how to fix it.

2. Concept / Understanding

- **IDOR** (Insecure Direct Object Reference) is an access control vulnerability. The application uses a reference (like an id number) to decide what data to return, but it does not check if the user is allowed to see that data.
- **Authentication** is proving who I am (username and password, then a cookie or token on every request). **Authorization** is checking what I am allowed to do. Authorization needs authentication first.
- **Horizontal privilege escalation:** getting the data of another user with the same level of access. Most IDOR cases are like this. (Vertical is getting a higher role, like admin.)
- **Obvious references:** a sequential number like user_id=10 is easy to guess. Changing it to 11 can show another user's data.
- **Hidden references:** the id can be encoded or hashed. For example, base64 *Mg*== is just the number 2, and values that look like MD5 or SHA1 hashes can also be created from the number. These do not protect the data.
- **How to spot it:** look in the browser Developer Tools (Network and Storage tabs) for ids in requests, URLs and local storage, then change them and watch the response.
- **The fix:** check on the server, on every request, that the logged-in user owns the object. Hard-to-guess ids help, but they are not a replacement for this check.

3. Practical Work

I finished the room 100%. The target was the TrypresentMe parent dashboard, and I did the following:

#	Step	What I found
1	Logged in to the app as the parent **niels** (credentials from the room).	The dashboard shows my account information and my children.
2	Opened Developer Tools, Network tab, and refreshed the page.	A request `GET /api/parents/view_accountinfo?user_id=10`. The user_id 10 is my own user, so the application uses it as the reference.

3	Opened Storage, Local Storage.	The key *auth_user* stores `{"user_id":10, ...}`. The request is built from this value.
4	Changed the user_id to other values and checked the response.	The application returned the data of other users, so there is no ownership check (IDOR).
5	Tried different user ids to find the parent with 10 children.	The parent has **user_id 15**. The answer was accepted.

The answers to the room questions were accepted: IDOR stands for Insecure Direct Object Reference, and most IDOR cases are horizontal privilege escalation.

4. Tools Used

- Firefox on Kali Linux
- Browser Developer Tools (Network and Storage tabs)
- TryHackMe room target machine

5. Evidence

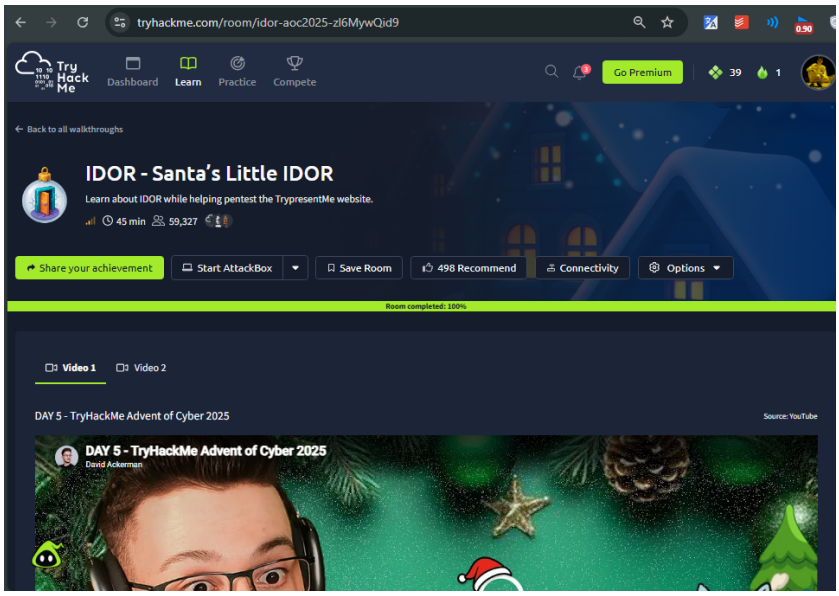


Figure 1: IDOR room page, 100% completed.

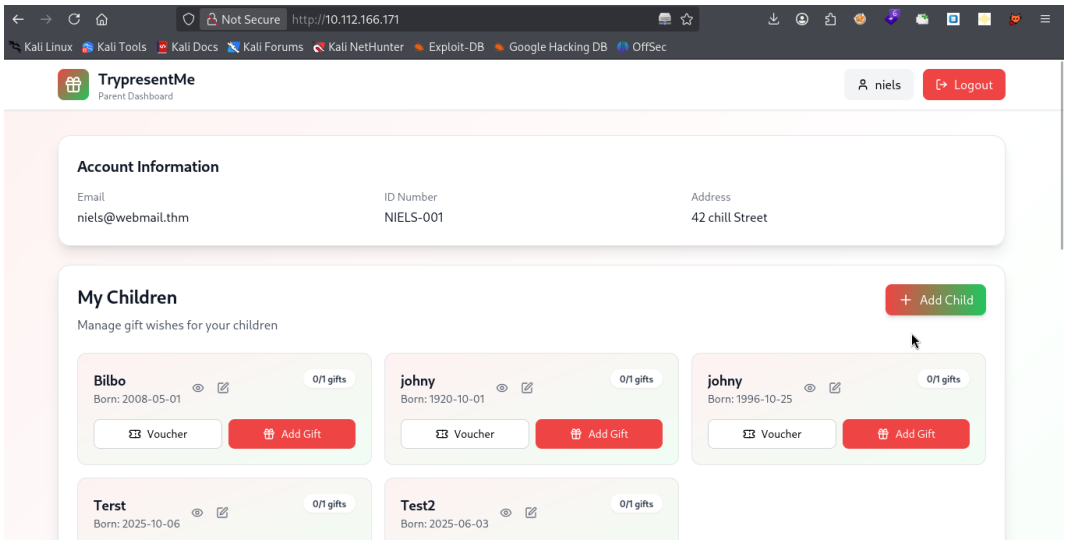


Figure 2: TrypresentMe parent dashboard, logged in as niels (my account and my children).

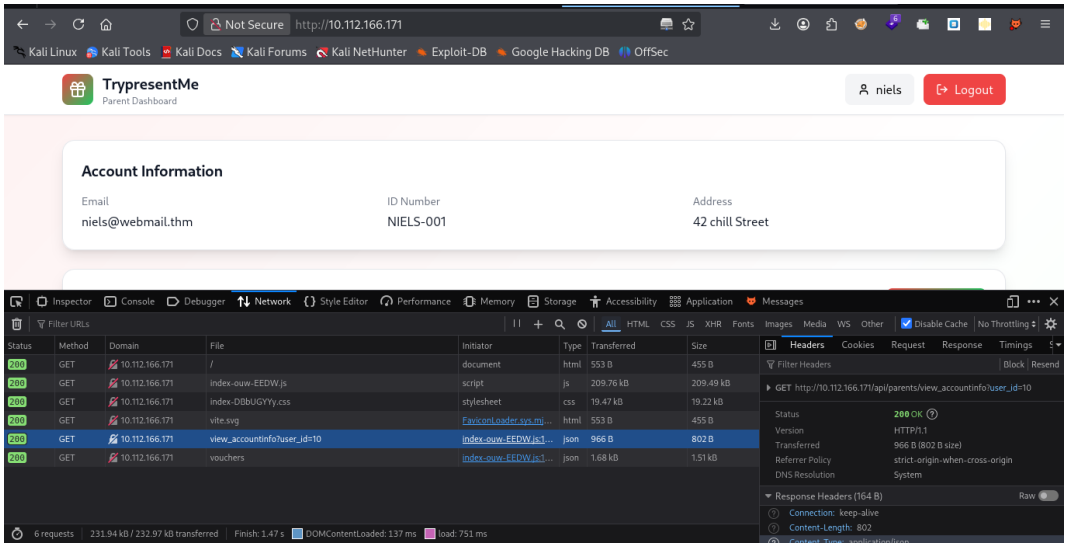


Figure 3: Network tab. The request view_accountinfo?user_id=10 returns 200 OK.

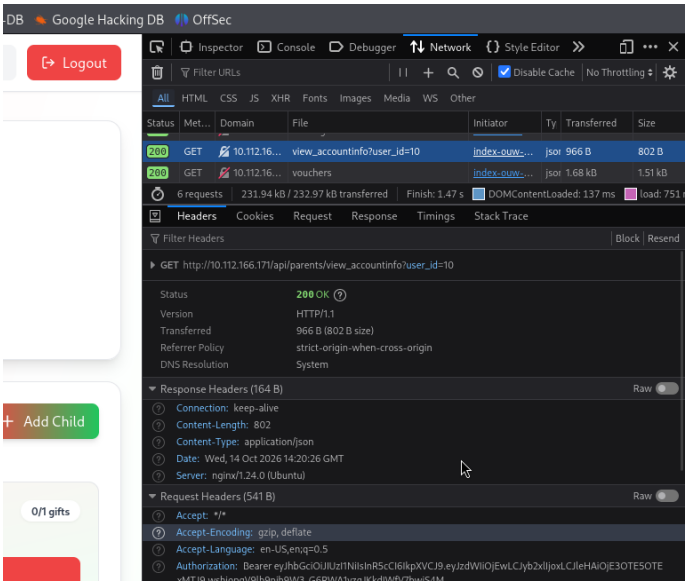


Figure 4: details of the view_accountinfo request. The user_id is sent in the URL, and the Authorization header carries my token.

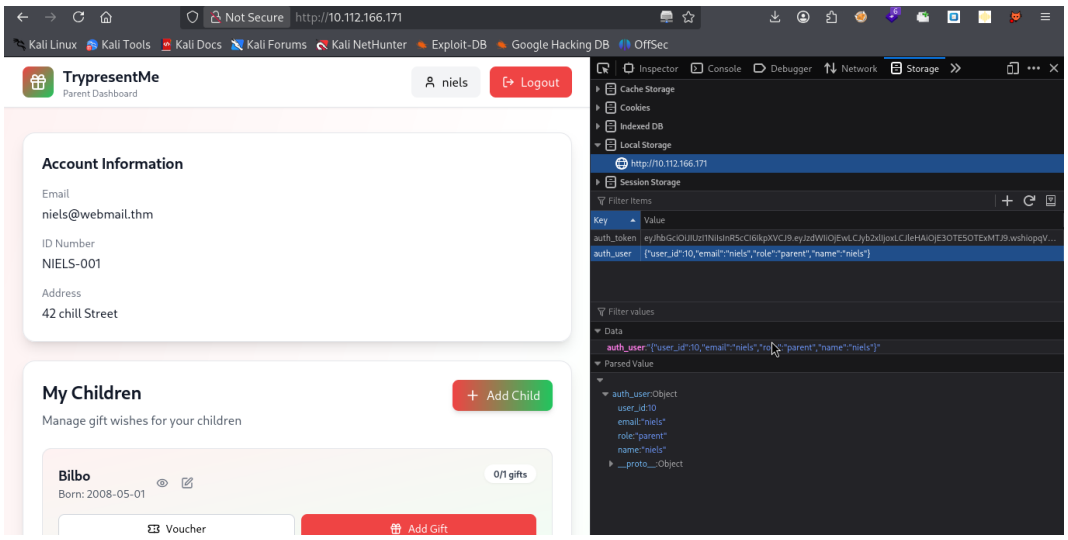


Figure 5: Local Storage. The key auth_user holds user_id 10, which the application uses as the reference.

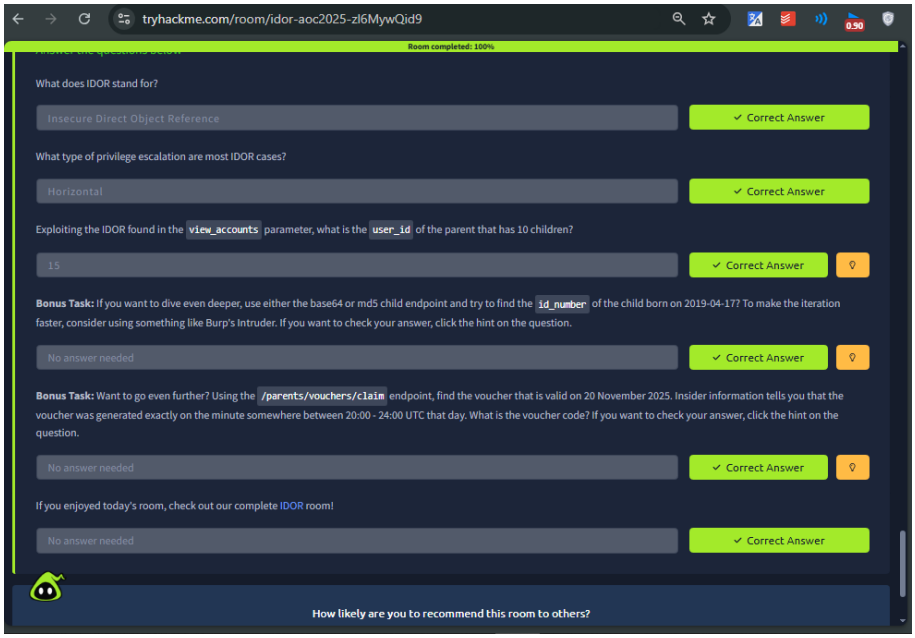


Figure 6: the room questions, all answers accepted (including user_id 15).

6. Result

Completed the room 100%. I found that the application trusts the user_id sent by the client, and by changing it I could get the data of other users. I found the parent with 10 children (user_id 15).

7. Key Learnings

- IDOR happens when the server does not check if the user owns the object that is requested.
- Ids in URLs, requests and local storage are controlled by the user, so they cannot be trusted.
- Encoding (base64) or hashing an id does not protect it.
- Authentication and authorization are different: the server must check both on every request.

8. Impact and Remediation

Impact: a logged-in user can read the personal data of other users (emails, addresses, children's details) just by changing an id. Depending on the endpoint, data could also be changed, and with ids that are easy to guess, an attacker can go through all the users.

Remediation:

- Check authorization on the server for every request: the object must belong to the user in the session token, not to the id in the request.
- Do not trust values stored on the client side (like user_id in Local Storage).
- Use unpredictable ids (like UUIDs) as an extra layer, not as the only protection.
- Log and rate-limit requests that go through many ids.